Resort Empire — Build dokument: Prototyp → MVP

Projekt: Resort Empire: Paradise Tycoon **Zdroj pravdy:** Verzia IV. Verzia II/III = len referencia.
Fáza: prototypovanie — cieľ je overiť pocit, nie vybalancovať ekonomiku.

> **Ako čítať:** §1–2 sú spoločné (fázový model, zamknuté rozhodnutia a stav otvorených bodov). §4 je celý prototyp. §5 je MVP. Každá fáza má scope v kusoch (producent), state machine (programátor), screeny (UI), čísla (ekonóm) a exit criteria (QA). Čísla sú `e.g.` (ilustrácia) alebo `TUNING` (východisko na ladenie) — žiadne nie je finálne.

---

1. Fázový model

Fáza	Čo dokazuje	Rozsah	Exit criteria
Prototype	Wave loop *pocitovo* funguje: vidím problém → rozhodnem → zlepším	1 typ hosťa; štart 3 budovy / 3 needy / **2 `Worker`-i** → koniec 5 / 5 / **4** (dva unlocky: `Pool`, `Bungalows`), 8 vĺn, 6 screenov, len wave loop	Hráč rozumie, že manažuje systém; **alokácia `Worker`-ov = rozhodnutie, nie lookup**; upgrade = zaslúžený; unlock = chuť pokračovať. Overené playtestom.
MVP	Okolo overeného core loopu drží *hra*	+ VIP vrstva, + Reputation, + obsah, + monetizačné háčiky	Core loop drží pri obsahu; VIP = spike moment; háčiky netrestajú; hráč sa chce vrátiť.
Next	*(uvidíme po MVP)*	TBD	TBD

Prečo tento split: prototyp je zámerne malý — testuje *pocit* core loopu, nie ekonomiku ani obsahovú šírku. Všetko, čo netestuje wave loop (VIP, Reputation, meny, monetizácia), je odložené do MVP, aby playtest nemal viac premenných, než dokáže rozlíšiť.

---

2. Zamknuté rozhodnutia

Kompletný záznam rozhodnutí a zmien (s dátumami) je v samostatnom **Decision Log** (companion dokument). Tu je len to, čo priamo tvaruje build:

Hard rule naprieč oboma fázami: **Perfect Stay / 5★ musí byť dosiahnuteľný bez minutia Energy.** Ak sa poruší, Energy sa potichu stala gate-om.

Zhrnutie zamknutých rozhodnutí, ktoré nesie tento dokument:

- Energy = optional boost (nie gate); earning = drop z vlny/VIP + rewarded video, žiadny regen.
- Resort Value = status (len rastie); Reputation = threshold unlock mena (nikdy neklesá).
- Guest Request Board = VIP only; VIP sa rieši pripravenosťou (Fix/Speed Up), nie staffom. S3 (alokácia `Worker`-ov) je len pre bežnú vlnu.
- Prototyp testuje jednu vec (core wave loop feel), NEtestuje ekonomiku/retention/monetizáciu/VIP.
- Prototyp má **dva** unlocky: `Pool` (need `Relax`) po `Wave 3` a `Bungalows` (need `Ocean View`) po `Wave 6`; každý prinesie `Worker`-a navyše. Demand prichádza postupne; vlna sa pri zatvorení pauzne.
- Unlock je dvojkrokový: eligibility gate sprístupní (prototyp míľnik / MVP Reputation), Coins kúpia.
- `Worker` prichádza v balíku s budovou (deficit ostáva 1); nový need debutuje mäkko (§4.8).
- Energy detaily: prototyp bez capu, žiadny refund, max 1 boost na budovu za vlnu, Mode A prototyp / Mode B MVP. Vo vlne boost dočasne zrýchli budovu (§4.6).
- Vlna: rovnomerný príchod `90 s`, potom dobeh do prázdnych radov; hodnotí sa jednotka needu; dve podlahy; najhorší výsledok `1★` (§4.4). Hard pravidlá wave systému WR1–WR9 (§4.8).
- Príjem: `Coins = jednotky vlny × coins_per_unit × násobič ratingu` (§4.8).
- MVP: `Manager` a `Specialist` sa najímajú za `Coins`; `Worker`-i so stropom `80 %` slotov; `Spa` = VIP budova; rada `Lv1–5` nadväzuje na prototyp (§5.3, §5.4, §5.6).

Všetky otvorené body zo série D1–D9 sú zavreté, rovnako míľniky unlockov, tematická deliaca čiara `Reception`/`Bungalows` aj staff model (prototyp = `Worker` alokácia §4.5, MVP = dvojvrstvový model §5.3). **Žiadny otvorený dizajnový bod neblokuje build prototypu.** MVP má tri otvorené body (Mode B vo vlne, VIP thresholdy, rada `Spa`) — viď §5.8.

Zvyšok sú tuning-otázky pre ekonóma/QA po teste (viď §4.8, §5.8) — z nich najkritickejšie sú `service_time` a `utilization_empty`, lebo od nich závisí, či prototyp vôbec niečo testuje. Každá zmena sa overuje nástrojom `wave_sim.py` proti WR7 a WR8.

4. PROTOTYPE

4.1 Cieľ fázy a čo testujeme

Prototyp existuje na zodpovedanie jednej otázky: **je wave loop pocitovo dobrý?**

Konkrétne testujeme:

- Pochopí hráč, že manažuje **systém**, nie jednotlivých hostí? (nesmie začať ťahať hostí prstom)
- Je „vidím bottleneck → presuniem `Worker`-a → problém sa zlepší" **satisfying**? Čiže: **je alokácia naozaj rozhodnutím**, alebo má vždy zjavnú odpoveď?
- **Prečíta hráč z panelu, či má investovať do budovy alebo presunúť ľudí?** Rad pri budove vyzerá rovnako, či je príčina „nikto neobsluhuje" (→ presun, zadarmo, hneď) alebo „budova je primalá" (→ upgrade, stojí `Coins`, až po vlne). Ak to hráč nerozozná, háda — a celý model sa zosype v UI, nie v mechanike. Toto sa nedá zistiť inak než na ľuďoch.
- Cíti sa upgrade ako zaslúžený následok konkrétneho problému, nie ako náhodné číslo?
- Dáva „dokončená vlna + jeden upgrade" prirodzený koniec krátkej session?
- Vytvorí odomknutie novej oblasti chuť pokračovať — a vydrží to aj pri **druhom** unlocku? (`Pool` po `Wave 3`, `Bungalows` po `Wave 6`. Prvý unlock poteší takmer vždy; druhý hovorí, či je to rytmus alebo jednorazový efekt.)

Čo **NE**testujeme: ekonomiku, monetizáciu, retention, VIP, obsahovú hĺbku. Čísla sú throwaway.

4.2 Scope v kusoch (producent)

Prvok	**Počet v prototype**
Typ hosťa	1 (`Casual Tourist`, ako demand, nie individuálny)
`Worker`	anonymný pool, **vždy o 1 menej než budov**: 2 → 3 (po `Pool`) → 4 (po `Bungalows`). Presun medzi budovami zadarmo a okamžite.
`worker_slots`	1 na budovu (všetky budovy, celý prototyp) — viac slotov prichádza až v MVP

Prvok	Počet v prototype
Menovaný staff (`Chef`, `Bartender`…)	**žiadny.** V prototype neexistuje ani ako herná entita, ani ako postavička v budove — viď §4.5. `Manager` vrstva prichádza až v MVP.
Budovy	5 celkom — **3 od začiatku** (`Reception`, `Restaurant`, `Beach Bar`), **2 ako unlock** (`Pool` po `Wave 3`, `Bungalows` po `Wave 6`)
Need typy	5 celkom (`Rooms`, `Drinks`, `Food` od začiatku; `Relax` po unlocku `Pool`, `Ocean View` po unlocku `Bungalows`)
Vlny	8 skriptovaných (nie procedurálne) — dva unlocky potrebujú miesto na debut a na jednu plnú vlnu po ňom
Screeny	6 (S1–S5 + S9 Area Unlock, viď UI handoff)
Meny	2 (`Coins`, `Energy`) + status `Resort Value`
Mini-event	1 (`Sudden rush` — nával na práve neobsadenú budovu, vlna 2, US15)

Žiadne: Reputation, Materials, Gems, VIP, Request Board, klub, event hub, IAP, menovaný staff, `Manager` vrstva, staff acquisition/rarity/leveling.

4.3 Core loop — state machine (programmer)

```
None
AWAITING_WAVE ──player: Start Wave──► WAVE_INCOMING ──guests spawn──► WAVE_ACTIVE
   ▲                                                                        │
   │                                          wave_timer uplynul AND všetky
backlog = 0
   │                                                                        ▼
   └── UPGRADE ◄────player: Claim──── WAVE_COMPLETE ◄──────────────────────────
WAVE_RESOLVING
        │  (optional)
        └──player: skip/confirm──► AWAITING_WAVE  [next wave unlocked, larger]

WAVE_ACTIVE ──app backgrounded──► WAVE_PAUSED ──app resumed──► WAVE_ACTIVE
```

- `AWAITING_WAVE`: resort v stave `Ready for Guests`, aktívne CTA `Start Wave`. Hráč vidí **forecast** ďalšej vlny a **môže presúvať `Worker`-ov** ešte pred štartom (§4.5). Upgrady sú dostupné.
- `WAVE_INCOMING`: animácia lode/shuttle, hostia sa spawnujú (transient, ~2–4s).
- `WAVE_ACTIVE`: má dve fázy, hráč v oboch robí to isté (presúva `Worker`-ov, používa Energy boost):
  - **príchod** (`0 → wave_timer`, `e.g.` 90 s): demand prichádza rovnomerne, needy sa plnia, vo vlne 2 sa spustí mini-event (§4.6);
  - **dobeh** (po `wave_timer`): nový demand už neprichádza, budovy obsluhujú, čo zostalo v radoch. Trvá, kým nie sú všetky `backlog = 0`. Upgrady v `WAVE_ACTIVE` **nie sú** dostupné (§5.3.7).
- `WAVE_PAUSED`: appka na pozadí/zatvorená. **Všetky timery zamrznuté, celý stav uložený.** Nič nepostupuje, nič sa nezhoršuje. Po návrate hráč pokračuje presne tam, kde skončil.
- `WAVE_RESOLVING`: spustí sa, keď je príchod uzavretý **a** všetky needy majú `backlog = 0`. `wave_timer` sám vlnu **neukončí** — len zatvorí príchod (rozhodnuté 2026-09-29).
- `WAVE_COMPLETE`: výsledková obrazovka (rating, odmeny, breakdown).
- `UPGRADE`: voliteľný odporúčaný upgrade; preskočiteľný. Area unlock (S9) sa spustí tu, keď je splnený niektorý z dvoch gate-ov (`Pool` po `Wave 3`, `Bungalows` po `Wave 6`). **Ak hráč unlock hneď nekúpi** (nemá dosť `Coins` alebo zvolí `Later`), stavenisko na mape ostane označené ako `available` a **ťuknutím naň v `AWAITING_WAVE` sa S9 otvorí znova**. Kým unlock nie je kúpený, jeho need sa nespawnuje (WR9).
- Návrat do `AWAITING_WAVE` s dostupnou ďalšou (väčšou) vlnou.

  Terminológia: pokojový stav je `AWAITING_WAVE`, nikdy `IDLE`. Slovo „idle" je zakázané vo všetkých materiáloch projektu (rules §6) — nesmie vojsť do kódu ako názov stavu.

  Prečo pauza, nie dobiehanie v reálnom čase: vlna je aktívny manažérsky moment. Jej beh offline by porušil rules §7 („vlny, ktoré prebehnú bez hráča" — nepovolené) a hráčovi by dal výsledok, ktorý neriadil. Dlhé *služby* spustené pred odchodom sa môžu dokončiť v reálnom čase (rules §7, povolené) — to je VIP/MVP záležitosť, nie wave.

4.4 Wave & needs model (programmer)

Príchod demandu — ako vzniká bottleneck

Demand **prichádza postupne počas vlny**, nie naraz. Každý need má celkový `demand` na vlnu, uvoľňovaný v čase rýchlosťou `arrival_rate`. Bottleneck existuje, keď `arrival_rate > fill_rate` pre daný need — backlog rastie.

Dôvod: keby celý demand prišiel pri spawne, backlog by kulminoval v t=0 a už len klesal — hráč by videl jeden problém na začiatku a nič potom. Postupný príchod vytvára napätie *počas* vlny a umožňuje, aby sa bottleneck objavil v strede vlny (US13: „Restaurant už zvláda, ale Beach Bar začína nestíhať").

Každý **need** v aktívnej vlne drží:

Pole	**Typ**	**Význam**
`need_id`	enum	Rooms / Drinks / Food / Relax / Ocean View
`demand_total`	int	celkový počet jednotiek, ktoré vlna vyžiada
`arrival_rate`	float	jednotiek/s uvoľnených do `demand_arrived` (`TUNING`)
`demand_arrived`	int	doteraz vyžiadané jednotky (≤ `demand_total`); rastie rýchlosťou `arrival_rate`
`filled`	int	doteraz obslúžené jednotky
`fill_rate`	float	jednotiek/s, ktoré služba vyprodukuje = `capacity × utilization ÷ service_time`; dvíha ju pridelený `Worker` alebo vyšší level budovy
`capacity`	int	**počet súbežných miest** v obsluhe (z levelu budovy) — koľko hostí sa obsluhuje NARAZ, nie za sekundu
`service_time`	float	sekundy na obslúženie jedného hosťa (`TUNING`) — rýchlosť je samostatný parameter, nie súčasť `capacity`

Pole	Typ	Význam
`backlog`	int	odvodené: `demand_arrived - filled`

Príchod a koniec vlny (rozhodnuté 2026-09-29)

Príchod je rovnomerný: `arrival_rate = demand_total ÷ wave_timer` pre každý need, konštantný počas celého príchodu. Jedinou výnimkou je mini-event `Sudden rush` (§4.6), ktorý pridá krátky nával navyše. Rytmus vlny (pokojný štart, špička, doznievanie) prototyp zámerne nemá — ak sa ukáže, že vlna pôsobí jednotvárne, dá sa pridať bez zmeny zvyšku systému.

`wave_timer` (`e.g.` 90 s, `TUNING`) ukončí PRÍCHOD, nie vlnu. Po ňom noví hostia nechodia a budovy dobiehajú rady. Vlna sa uzavrie, až keď sú všetky rady prázdne.

Prečo: rozhodli sme, že nikto neodíde počas vlny a rad sa vždy dobehne. Tvrdý koniec by obe pravidlá porušil (ľudia v rade by fakticky odišli neobslúžení) a vrátil by posledné sekundy do režimu „nestíham" (non-goal). Takto žiadni neobslúžení neexistujú.

Vlna je vždy konečná, lebo neobsadená budova nikdy nebeží na nule (AC2c). Dobre zvládnutá vlna trvá `e.g.` ~2 minúty (príchod 90 s + dobehnutie posledných obsluh); zle zvládnutá sa predĺži — pasívna hra v neskorých vlnách aj o niekoľko minút (viď OPEN o dĺžke dobehu v Decision Logu). Nie je to trest navyše: jednotky v rade už čakajú dlho a satisfaction to zachytí. Dôsledok pre ekonóma a QA: **vlna nemá pevnú dĺžku.**

Ako budova obsluhuje — miesta pri pulte (rozhodnuté 2026-09-29)

Budova má `capacity` miest. Jednotka needu (hosť so svojou potrebou) príde, sadne si na voľné miesto a obsluha trvá `service_time ÷ utilization` — s `Worker`-om `12 s`, bez neho `12 ÷ 0.33 ≈ 36 s` (`e.g.`). Keď sú všetky miesta obsadené, ďalší stoja v **rade**.

Čakanie = čas od príchodu po usadenie. Čas, keď jednotka sedí a obsluhuje sa, sa **nepočíta** — to je služba, nie čakanie. (Keby sa počítal, pri `service_time 12 s` a `patience_grace 10 s` by nikto nikdy nedostal 100 % a Perfect Stay by bol nemožný — HR2 by padlo.)

Prečo miesta a nie plynulý tok: je to presne to, čo hráč vidí (stoličky, ľudia na nich, rad za nimi), a programátor to takto postaví prirodzene. Model musí byť jeden — obe rozumné implementácie dávajú citeľne iný balans, a ekonóm musí ladiť ten, ktorý je v hre. `wave_sim.py` počíta presne tento model.

`fill_rate = capacity × utilization ÷ service_time` ostáva ako **priemerná** priepustnosť (na odhady a tabuľky); skutočný beh je po miestach.

Obsluha je priebeh, nie pevný časovač (rozhodnuté 2026-09-29). Každá rozbehnutá obsluha postupuje rýchlosťou `utilization ÷ service_time` (v MVP krát `manager_multiplier`, pri boostu krát `1 ÷ (1 − energy_speed_reduction_pct)`). **Keď sa rýchlosť zmení — príde alebo odíde `Worker`, začne alebo skončí Energy boost — zmena platí okamžite aj pre hostí, ktorí už sedia.** Dôvod: bez toho by presun `Worker`-a nemal viditeľný efekt, kým sa nedokončia rozbehnuté obsluhy (pri prázdnej budove až ~36 s), a AC2 („obsluha sa okamžite zrýchli") by sa nedalo splniť. Rovnako odchod `Worker`-a spomalí aj rozbehnuté obsluhy — cena presunu je okamžite viditeľná.

Satisfaction — akumuluje sa počas vlny, vyhodnocuje sa až na konci (rozhodnuté 2026-09-29)

Wave Satisfaction sa NEpočíta zo stavu na konci vlny, ale z histórie čakania. Počas vlny sa ticho zbiera, ako dlho čakala **každá jednotka needu**; na `Wave Complete` sa z toho spočíta jedna hodnota 0–100 %.

Jednotka hodnotenia = jednotka needu, nie hosť (rozhodnuté 2026-09-29). Každá jednotka `Drinks`, `Food`… má vlastné čakanie a vlastný príspevok; hosť je len vizuálna postavička, ktorá needy nesie. Hra nesleduje, ktorý konkrétny hosť čo chcel.

```
None
Wave Satisfaction = max( wave_floor ,  Σ (príspevok_jednotky × váha_needu) ÷ Σ váha_needu )
```

Váha je `1.0` pre objemové needy a vyššia pre `Ocean View` (`ocean_view_satisfaction_weight`, `TUNING`). Prečo jednotka a nie hosť: počíta sa presne to, čo hráč vidí — čakanie v rade pri konkrétnej budove. Model „celý hosť" by musel rozhodnúť, ako spojiť viac needov jedného hosťa (priemer? najhorší? prvý?), a každá odpoveď je iná hra; zároveň by vnášal jednotlivých hostí späť do hry, čo pilier 5 vylučuje. Individuálne príbehy hostí patria VIP.

Dôvod: keby sa hodnotil koncový *stav* (snapshot — „koľko radov je prázdnych"), optimálna stratégia by bola vlnu ignorovať a dobehnúť ju v posledných sekundách. Čakanie by nestálo nič a manažment počas vlny by stratil zmysel. Akumuláciou platí oboje naraz: **priebeh je pokojný (hráč nevidí tikajúce skóre), ale rýchlosť reakcie rozhoduje o hviezdičkách.**

Čakanie jednotky	Príspevok k satisfaction
krátke (do `patience_grace`, `e.g.` 10 s)	**100 %** — rýchla náprava = plný rating
stredné (`e.g.` 10–30 s)	klesá lineárne `100 % → 25 %` (`TUNING`)
dlhé (nad `e.g.` 30 s)	drží sa na **podlahe jednotky `25 %`** (`TUNING`)

Dve podlahy — hráč nikdy nedostane nulu (rozhodnuté 2026-09-29):

Podlaha	Hodnota	Čo chráni
na jednotku (`unit_floor`)	`25 %` (`TUNING`)	hráča, ktorý zaostal v prvej tretine, pred nedobehnuteľnou vlnou
na vlnu (`wave_floor`)	`40 %` (`TUNING`) = `1★`	pred prázdnym `Wave Complete` — aj najhoršia vlna dá `1★` a nejaké `Coins`

> **Pevné pravidlo medzi podlahami (spec, neladí sa preč): `unit_floor` < `wave_floor` < hranica `3★`.** Ak je podlaha jednotky vyššia než podlaha vlny, priemer nikdy neklesne pod ňu: podlaha vlny je mŕtva, spodné hviezdičky nedosiahnuteľné a hráč, ktorý **počas vlny nespraví nič**, dostane slušný rating. Presne to sa stalo pri pôvodných hodnotách `60 % / 40 %` (viď CHANGED v logu).

Rating (`TUNING`): `1★ 40 % · 2★ 55 % · 3★ 70 % · 4★ 80 % · 5★ 90 % · Perfect 100 %`. Najnižší dosiahnuteľný výsledok je `1★` (podlaha vlny).

> **Pozor pri ladení:** podlahy sa nemajú overovať otázkou „nie je to príliš kruté?", ale opačnou — **„je vôbec cítiť rozdiel medzi 3★ a 5★?"** Ak je podlaha privysoká, nedbalá a starostlivá hra dopadnú podobne a hráč nemá dôvod reagovať rýchlo. Patrí do exit criteria playtestu.

Žiadne viditeľné odpočty trpezlivosti a žiadne živé skóre. Akumulácia je čisto vnútorná veličina — nad hlavami hostí nie sú bublinky s klesajúcim časovačom a `Wave Satisfaction` sa počas vlny **nezobrazuje** ako číslo. Hráč vidí **rad** a **warning**; to je jeho signál. Číslo uvidí až na `Wave Complete`.

Hostia počas vlny NEODCHÁDZAJÚ (rozhodnuté 2026-09-29)

Počas vlny neodíde nikto, bez ohľadu na to, ako dlho čaká. Rad sa vždy dobehne, vlna sa vždy dá dokončiť. Nespokojnosť sa prejaví **až na `Wave Complete`**: časť hostí odchádza s tým, že im pretiekli nervy.

Koľko postavičiek odíde, sa odvodí z celkového `Wave Satisfaction`, nie z osudu konkrétnych ľudí (hra jednotlivých hostí nesleduje). `e.g. 5★` = nikto, `3★` = pár, `1★` = viditeľný hlúčik. Presné mapovanie je vizuálne rozhodnutie, nie mechanika.

Dôvod: odchádzajúci hosť uprostred vlny je moment, po ktorom hráč vie, že to už nenapraví — a to je presne „prehral si", ktorý je non-goalom. Ako post-wave obraz nesie tú istú informáciu mäkším nosičom: hráč nevidí trest, vidí **ako dopadol jeho rezort**.

> **Odchod nesmie mať mechanický dopad na ďalšiu vlnu.** Je to *vyjadrenie* ratingu, nie druhý trest navrch. Keby odídení hostia zmenšovali budúce vlny alebo krátili odmeny nad rámec ratingu, vznikla by špirála: zlá vlna → menší príjem → horšia ďalšia vlna.

Ako to beží v čase

Fáza	**Čo sa deje**	**Čo hráč vidí**
príchod (0–90 s)	jednotky prichádzajú rovnomerne, čakanie sa ticho akumuluje	rad rastie, warning, guest toast
dobeh (po 90 s)	noví nechodia, budovy dobiehajú rady	rady sa zmenšujú; hráč stále môže presúvať a boostovať
koniec vlny	z akumulácie sa spočíta `Wave Satisfaction`	rating, breakdown
po vlne	časť hostí odchádza podľa spokojnosti	„niekoľkým pretiekli nervy" — obraz výsledku
medzi vlnami	rewards → upgrade	S5

Stav jedného needu (rozhodnuté 2026-09-29)

Počíta sa len rad čakajúcich (`queue`) — jednotky, ktoré ešte nesedia. Ľudia na miestach sa nepočítajú: obsadená budova má počas vlny takmer stále plné miesta a to je zdravý stav, nie problém.

```
None
OK ——————queue > bottleneck_threshold AND rastie ≥
bottleneck_hysteresis_s——————▸ BOTTLENECK
BOTTLENECK ——queue sa zmenšuje——▸ RECOVERING
RECOVERING ——queue ≤ bottleneck_threshold——▸ OK
RECOVERING ——queue znova rastie ≥ bottleneck_hysteresis_s——▸ BOTTLENECK
OK / RECOVERING ——príchod uzavretý AND queue = 0 AND nikto nesedí——▸ SATISFIED
```

Stav	Čo hráč vidí
`OK`	pokoj
`BOTTLENECK`	`⚠ Beach Bar queue growing` + guest feedback toast
`RECOVERING`	pozitívny signál, napr. `Beach Bar is catching up`
`SATISFIED`	need hotový

Prečo `RECOVERING`: po presune `Worker`-a hráč dostane **okamžitý a pravdivý** dôkaz, že rozhodnutie zabralo (US8) — a zároveň vidí, že budova ešte nie je hotová, takže ho nezláka vziať `Worker`-a hneď späť. Keby warning zmizol hneď, obrazovka by hlásila pokoj, kým pri budove stojí rad, a pri prvom návrate `Worker`-a by warning vyskočil znova. `bottleneck_hysteresis_s` (`e.g.` 3 s, `TUNING`) bráni blikaniu, keď rad kolíše okolo prahu. `Relax` ostáva `LOCKED` (vôbec sa nespawnuje), kým sa neodomkne `Pool`; `Ocean View` rovnako, kým sa neodomknú `Bungalows`. `LOCKED` need sa nezobrazuje na S2 a negeneruje demand.

`Ocean View` je flow need, nie vlastnosť budovy (rozhodnuté 2026-09-29). Hostia ho žiadajú počas vlny — obsluha ich sprevádza a usádza v bungalove s výhľadom — takže má `arrival_rate`, `fill_rate` aj `backlog` ako každý iný need. Dva rozdiely oproti objemovým needom:

Vlastnosť	Hodnota	Dôvod
`ocean_view_guest_share`	`0.30` (`e.g.`) — podiel hostí, ktorí ho generujú	premium potreba, nie univerzálna; nízky objem. **V prototype len pomôcka** pri tvorbe tabuľky vĺn — objem sa zadáva priamo v tabuľke.

Vlastnosť	Hodnota	Dôvod
`ocean_view_satisfaction_weight`	vyššia než objemové needy (`TUNING`)	bottleneck tu bolí viac, hoci je menší

Dôvod tohto tvaru: piaty need s rovnakou mechanikou a rovnakým objemom by z S2 spravil tabuľku (non-goal „15 statov na obrazovke"). Nízky objem + vysoká váha pridáva nový **typ** rozhodnutia — „nechám premium potrebu chvíľu čakať, alebo obetujem objemový need?" — namiesto ďalšieho merača rovnakého druhu.

4.5 Worker alokácia & bottleneck resolution

Prototyp testuje alokáciu objemu, nie priraďovanie menovaných ľudí (rozhodnuté 2026-09-29). Je to spodná vrstva MVP modelu (§5.3) bez hornej — `Worker`-i áno, `Manager`-i nie.

Ako to funguje

- **`Worker` je anonymný a zameniteľný.** Nemá meno, špecializáciu ani level. Presun medzi budovami je **zadarmo a okamžitý**, kedykoľvek počas vlny.
- **Budova s `Worker`-om beží naplno, bez neho spomalene.** `utilization` je `1.0` s obsluhou a `e.g. 0.33` bez nej (`TUNING`) — prázdna budova nie je mŕtva, len pomalá.
- **Každá budova má 1 slot.** Nedá sa nasypať viac ľudí do jednej budovy; viac slotov prichádza až v MVP (§5.3.6, `Lv4`).
- **`Worker`-ov je vždy o jedného menej než budov.** Aspoň jedna budova je vždy neobsadená — a to je zdroj rozhodnutia.
- **`Worker` vždy stojí na niektorej budove** — neexistuje „lavička" nepriradených. V prototype je preto vždy **presne jedna** budova prázdna, a presun znamená „vyber, ktorú budovu nechám prázdnu". Na tom stojí aj mini-event (§4.6).

Fáza	Budovy	`Worker`	Rozhodnutie hráča
štart	3	**2**	ktorá služba ostane teraz bez nikoho
po `Pool`	4	**3**	to isté, o jednu možnosť viac
po `Bungalows`	5	**4**	to isté, plná zostava

Nový `Worker` prichádza s novou budovou, takže deficit ostáva konštantne 1 a pravidlo *„hráč nikdy nemá need bez nástroja"* platí ďalej — nová služba je vždy obslúžiteľná.

`Worker`-i medzi vlnami (rozhodnuté 2026-09-29)

- **Rozmiestnenie sa pamätá.** Kde hráč `Worker`-ov nechal, tam začnú ďalšiu vlnu.
- **Presúvať sa dá aj pred štartom** (`AWAITING_WAVE`). Hráč si pozrie forecast, rozmiestni ľudí a až potom ťukne `Start Wave`.
- Nový `Worker` z unlocku sa objaví **na novej budove** (tá by inak začínala prázdna).

Prečo: príprava podľa forecastu je presne pokojné rozhodovanie (pilier 2) a dáva forecastu zmysel. Zakázať presun pred štartom by z prvých sekúnd vlny spravilo preteky v klikaní; reset by zahodil prípravu a rezort by „zabúdal", čo hráč nastavil.

Vzniká tým krivka učenia, s ktorou treba počítať: vo vlne 1 stačí dobrý plán vopred (hráč sa učí čítať forecast a dostane Perfect bez jediného presunu počas vlny — to nie je chyba, viď metriky §4.12). Od vlny 2 plán nestačí — mini-event a needy nad hranicou ~7 si vynútia **plán aj reakciu počas vlny**. Debutové vlny (4, 7) sú zámerne mäkšie.

Alokačný flow

Objaví sa bottleneck → hráč tapne need → S3 → presunie `Worker`-a → `fill_rate` vyskočí → backlog sa vyčistí → pozitívny feedback toast. **Zároveň vidí, že budova, odkiaľ bral, ostala bez obsluhy** — alokácia je nulový súčet a bez viditeľnej straty nie je rozhodnutím.

Okamžitý viditeľný efekt je povinný (US8): obsluha sa zrýchli, warning sa zmení na `RECOVERING` (pozitívny signál), toast sa spustí. Bez toho loop pôsobí pasívne.

Menovaný staff v prototype NEEXISTUJE

Žiadny `Chef`, `Bartender`, `Receptionist`, `Lifeguard` ani `Bellhop` — ani ako herná entita, ani ako postavička stojaca v budove.

Dôvod: keby v bare stál barman aj bez prideleného `Worker`-a, „prázdna budova" by nevyzerala prázdno a **rozlíšenie *budova je malá* vs. *budova je neobsadená* by sa rozbilo** — a práve to rozlíšenie je jedna z testovaných otázok (§4.1). Budova bez `Worker`-a musí byť viditeľne bez obsluhy.

Menovaní špecialisti sa vracajú v MVP ako `Manager` vrstva (§5.3.2), kde majú vlastnú úlohu (`manager_multiplier`) a zberateľský zmysel. V prototype by boli kulisou, ktorá kazí čitateľnosť.

Mapovanie budova → need

Budova	Need	Dostupnosť
`Reception`	`Rooms`	od začiatku
`Restaurant`	`Food`	od začiatku
`Beach Bar`	`Drinks`	od začiatku
`Pool`	`Relax`	unlock po `Wave 3`
`Bungalows`	`Ocean View`	unlock po `Wave 6`

> **Tematická deliaca čiara `Reception` vs. `Bungalows`** (rozhodnuté 2026-09-29): `Reception` = check-in **a základné ubytovanie** — hotelový blok, kde hostia bývajú štandardne; obsluhuje `Rooms`. `Bungalows` = samostatné premium chatky pri oceáne; obsluhujú `Ocean View`. Deliaca čiara je **„strecha nad hlavou" vs. „zážitok z výhľadu"**. Preto je `Rooms` objemový need (každý hosť niekde spí) a `Ocean View` premium need s nízkym objemom (chce ho len časť hostí) a vyššou váhou (kto si ho vypýta, tomu na ňom záleží).

Čo budovy komunikujú (artist)

Nie ako majú vyzerať — čo musí hráč pochopiť na prvý pohľad, bez textu.

Budova	Čo komunikuje	Musí byť na prvý pohľad odlíšiteľné od
`Reception`	„tu sa hostia hlásia a tu bývajú" — funkčný hotelový blok, srdce prevádzky	`Bungalows` — Reception je jeden objem, Bungalows sú viacero drobných
`Restaurant`	„tu sa je" — sedenie, kuchyňa, jedlo na stoloch	`Beach Bar` — Restaurant má sedenie a strechu, Bar je otvorený pult
`Beach Bar`	„tu sa pije" — pult pri pláži, rýchla obsluha postojačky	`Restaurant`
`Pool`	„tu sa oddychuje" — voda, ležadlá, žiadna obsluha jedla	plážou (Pool je stavba, pláž je terén)

Budova	Čo komunikuje	Musí byť na prvý pohľad odlíšiteľné od
`Bungalows`	„tu sa býva luxusne" — samostatné chatky pri vode, výhľad na oceán	`Reception`

Prázdne plochy sú asset, nie diera. Na štarte sú na mape viditeľné miesta pre `Pool` a `Bungalows` — musia vyzerať ako *pripravené stavenisko*, nie ako nedokončená mapa. Hráč má mať od prvej sekundy kam pozerať a čo chcieť (US1).

Progresné rady: v prototype má každá budova **3 levely** (rozhodnuté, §4.8: `3 → 5 → 7` miest). Rozdiel musí byť viditeľný z odstupu na mobile a **počet miest musí byť z budovy vidieť** — je to jediná vec, ktorú upgrade mení, a pomáha rozlíšiť „primalá" od „neobsadená".

4.6 Energy v prototype — sekundárny prvok, nie predmet testu

Energy nie je to, čo v prototype meriame. Je prítomná preto, aby mini-event pôsobil živo — nie aby sme testovali jej ekonomiku.

Mini-event `Sudden rush` (rozhodnuté 2026-09-29)

- **Nával zasiahne budovu, ktorá je práve neobsadená.** V prototype je vždy presne jedna budova bez `Worker`-a (deficit 1), takže **primárne riešenie — presun `Worker`-a — existuje vždy**, z definície, nie náhodou. (Pôvodný `Beach Bar drink rush` pri obsadenom bare riešenie nemal — s jedným slotom nebolo kam presúvať.)
- **Objem a čas** (`e.g.`, `TUNING`): `+6` jednotiek needu tej budovy počas `10 s`, jedenkrát, vo **vlne 2** okolo `45 s`.
- **Text sa prispôsobí budove:** `Sudden rush at Beach Bar!` / `…at Restaurant!` / `…at Reception!` — jeden typ eventu, varianty textu.
- **Vlnu nepauzuje.** Výzva v S2 s tromi voľbami: `Move Worker here` (primárne — *toto testujeme*) / `Use Energy to serve faster` (bonus) / `Ignore`. Event musí byť plne zvládnuteľný presunom `Worker`-a.
- **„Vyriešený" nie je špeciálny stav.** Nával je len extra demand; vyrieši sa tým, že sa rad obslúži, a hodnotí sa rovnako ako všetko ostatné — čakaním.
- **Nával zvýši celkový objem needu** na paneli (`Drinks 9/16 → 9/22`) a počíta sa aj do odmeny (jednotky vlny). **Forecast ho neukazuje** — je to nečakaná situácia, na ktorú sa hráč nemôže pripraviť rozmiestnením.
- Vedľajší efekt: vlna 2 by pri pasívnej hre inak skončila na Perfect; nával z nej spraví prvý skutočný test alokácie bez zmeny tabuľky vĺn.

Energy boost

- Len Mode A. **Vo vlne je Mode A dočasné zrýchlenie jednej budovy** (rozhodnuté 2026-09-29): budova ukáže `Speed Up ⚡3`; po tapnutí sa jej `service_time` zníži na polovicu (`energy_speed_reduction_pct 0.50`, `TUNING`) na `15 s` (`energy_speed_duration_s`, `e.g.`), potom sa vráti na normál. **1 boost na budovu za vlnu.** Cena `⚡3` (`e.g.`).
- Boost nemení počet miest (HR7) a funguje v príchode aj v dobehu.
- **Boost je najsilnejší pri obsadenej budove** (s `Worker`-om zrýchli viac ľudí za sekundu než bez neho). Tým Energy alokáciu **dopĺňa**, nenahrádza — a keď v playteste hráči boostujú prázdne budovy, je to signál, že nepochopili alokáciu.
- Model „skráť zostávajúci čas služby" (`remaining_time`) platí len pre VIP služby (MVP), kde služba je jednorazová úloha s koncom. Vo vlne služby s odpočtom neexistujú.
- Získava sa: drop za dokončenie vlny podľa ratingu `[0,1,2,3,5]` (`TUNING`) + optional rewarded video.
- Pri `energy = 0`: tlačidlo ponúkne `Watch ad for ⚡`, nikdy neblokuje službu (HR1).
- **Uncapped; hoarding sa v prototype nerieši** (Energy nie je predmet testu). Cap a hoarding sa sledujú až v MVP.
- QA: **nemerať Energy usage ako success metriku prototypu.**
- Full detail: **see Energy spec (companion doc).**

4.7 Screeny (UI)

Prototyp = 6 screenov, definované v **UI Screens handoff (companion doc)**: `S1 Home/Island · S2 Wave Needs Panel · S3 Worker Allocation · S4 Wave Complete · S5 Upgrade · S9 Area Unlock (spustí sa 2×)`.

Kritický: **S2 je manažérsky dashboard, nie quest list.** To je prvok, ktorý najviac rozhoduje, či prototyp odovzdá „nie je to idle" pocit.

4.8 Čísla — východisko (všetko throwaway)

Tabuľka parametrov wave systému (programátor, ekonóm)

Všetky laditeľné hodnoty prototypu na jednom mieste. Hard pravidlá (spec) sú pod tabuľkou — tie sa ladením nesmú zmeniť. Parametre Energy sú v Energy spec §4.

Názov	Typ	Rozsah	Default	Kto ladí	Čo ovplyvňuje
`wave_timer`	float, s	45–180	`90` `TUNING`	ekonóm	dĺžka príchodu; vlna trvá dlhšie o dobeh
`arrival_rate`	float, jednotiek/s	odvodené	`demand_total ÷ wave_timer`	— (vzorec)	rovnomerný príchod
`demand_total`	int, per need per vlna	0–100	tabuľka vĺn `e.g.`	ekonóm	objem tlaku
`capacity`	int, miesta	1–15	`3 / 5 / 7` pre Lv1/2/3 `e.g.`	ekonóm	strop súbežnej obsluhy
`service_time`	float, s na jednotku	4–24	`12` `TUNING`	ekonóm	rýchlosť budovy — spolu s `utilization_empty` najcitlivejšie číslo; **v prototype rovnaký pre všetky budovy**
`utilization_staffed`	float	—	`1.0` (pevné)	—	budova s `Worker`-om
`utilization_empty`	float	0.1–0.6	`0.33` `TUNING`	ekonóm	**najkritickejšie číslo** — či je deficit rozhodnutím alebo trestom
`worker_slots`	int	—	`1` (pevné v prototype)	—	—

Názov	Typ	Rozsah	Default	Kto ladí	Čo ovplyvňuje
bottleneck_threshold	int, jednotiek v rade čakajúcich	1–10	4 e.g.	ekonóm	kedy sa ukáže warning (sediaci sa nepočítajú)
bottleneck_hysteresis_s	float, s	1–10	3 TUNING	ekonóm	ako dlho musí rad rásť, kým sa ukáže / vráti warning (proti blikaniu)
rush_wave / rush_time_s	int / float	—	2 / 45 e.g.	designer	kedy nastane mini-event
rush_units / rush_duration_s	int / float	—	6 / 10 e.g.	ekonóm	veľkosť návalu
patience_grace	float, s čakania (príchod → usadenie)	3–30	10 TUNING	ekonóm	do kedy je príspevok 100 %; čas obsluhy sa nepočíta
patience_floor_after	float, s	15–60	30 TUNING	ekonóm	od kedy platí unit_floor
unit_floor	float, %	0–40	25 TUNING	ekonóm	viď pravidlo nižšie
wave_floor	float, %	20–50	40 TUNING	ekonóm	najhorší možný výsledok = 1★
rating_thresholds	float[6], %	—	40 / 55 / 70 / 80 /	ekonóm	1★ … 5★, Perfect

Názov	Typ	Rozsah	Default	Kto ladí	Čo ovplyvňuje
			`90 / 100 TUNING`		
`ocean_view_satisfaction_weight`	float	1.0–3.0	`1.5 e.g.`	ekonóm	koľko bolí čakanie pri `Bungalows`
`ocean_view_guest_share`	float	—	`0.30 e.g.`	ekonóm	**len pomôcka pri tvorbe tabuľky vĺn** — v prototype sa objem `Ocean View` zadáva priamo v tabuľke
`new_need_debut_ratio`	float	0.3–0.8	`0.50 TUNING`	ekonóm	mäkkosť debutu nového needu
`demand_growth_per_wave`	float	0.1–0.5	`0.25 TUNING`	ekonóm	krivka obtiažnosti
`upgrade_cost`	int[2], `Coins`	—	`500 / 1,200 e.g.`	ekonóm	Lv1→2, Lv2→3
`unlock_cost`	int[2], `Coins`	—	`1,200 / 2,500 e.g.`	ekonóm	`Pool`, `Bungalows`
`coins_per_unit`	int	5–40	`16 e.g.`	ekonóm	príjem z vlny; spätne odvodené z cien (§ Príjem z vlny)
`coins_rating_mult`	float[6]	—	`0.4 / 0.6 / 0.8 / 1.0 / 1.2`	ekonóm	`1★ … 5★, Perfect`

Názov	Typ	Rozsah	Default	Kto ladí	Čo ovplyvňuje
			`/ 1.3 TUNING`		

Hard pravidlá wave systému (spec — NEladiteľné):

ID	Pravidlo
WR1	`unit_floor` < `wave_floor` < hranica `3★`. Inak nečinnosť dostane slušný rating.
WR2	`utilization_empty` > 0. Neobsadená budova nikdy nebeží na nule — inak vlna nemusí skončiť.
WR3	`wave_timer` zatvára príchod, nikdy nie vlnu. Vlna sa uzavrie až pri prázdnych radoch.
WR4	Jednotka počas vlny nikdy neodíde a nikdy neprispeje menej než `unit_floor`.
WR5	Presun `Worker`-a je vždy zadarmo a okamžitý (Energy HR6).
WR6	Upgrade nie je dostupný v `WAVE_ACTIVE` (§5.3.7).
WR7	**Od `Wave 3` (okrem debutových vĺn) najlepšie statické rozmiestnenie bez presunov, upgradov a Energy nesmie dosiahnuť `5★`.** Inak vlna netestuje nič. Debutové vlny (W4, W7) sú výnimkou, lebo majú byť zámerne mäkké. Overuje sa `wave_sim.py`.
WR8	**Plne pripravený rezort (všetky budovy na najvyššom leveli fázy) musí v každej vlne dosiahnuť `5★` bez Energy** (wave podoba HR2). Overuje sa `wave_sim.py`.
WR9	Need budovy, ktorú hráč ešte nekúpil, sa vo vlnách **nespawnuje** — ani keď je unlock

ID	Pravidlo
	sprístupnený. Tabuľka vĺn sa tým nelámie; stĺpec sa jednoducho neaktivuje.

Citlivosť kľúčových parametrov — čo sa stane pri 2× a 0,5× (rules §8):

Parameter	Metrika	Pri 2×	Pri 0,5×
`service_time`	`rating_distribution`, `drain_duration_s`	budovy pomalé; aj obsadené zaostávajú od vlny 2, vlny sa predlžujú na minúty, hra pôsobí beznádejne	budovy rýchle; obsadené nikdy nezaostanú, stačí jedno rozmiestnenie — **presne chyba pôvodných `6 s`** (porušuje WR7)
`utilization_empty`	`worker_moves_in_wave`	prázdna budova stíha → deficit nič neznamená, presuny zbytočné	prázdna budova takmer stojí → deficit je trest, dobehy veľmi dlhé
`unit_floor`	rozdiel `3★` vs. `5★`	ak prekročí `wave_floor`, nečinnosť dostane slušný rating (porušuje WR1)	zlé vlny padajú na `wave_floor` skôr; rozdiel medzi `1★` a `3★` sa zväčší
`patience_grace`	podiel `5★` a Perfect	tolerantné, Perfect ľahký, reakcia nemusí byť rýchla	prísne, Perfect takmer nedosiahnuteľný, tlačí k reflexom (pilier 2)
`coins_per_unit`	`upgrade_take_rate`, čas do unlocku	upgrady aj unlocky hneď, žiadne rozhodovanie čo kúpiť	`Pool` až po vlne 4–5, skriptovaná progresia sa rozpadá (WR9 zachráni tabuľku, nie zážitok)
`bottleneck_threshold`	kedy hráč reaguje	warning prichádza neskoro, hráč sa	warning pri každom malom rade, panel

Parameter	Metrika	Pri 2×	Pri 0,5×
		dozvie o probléme, keď už čakanie bolí	neustále „kričí", warning stratí váhu

Pravidlá rastu (rozhodnuté 2026-09-29)

Tri pravidlá, ktoré tvarujú každú progresnú tabuľku — platia aj keď sa čísla prepíšu:

1. **`Worker` prichádza v balíku s budovou.** Deficit ostáva konštantne 1 a hráč nikdy nevidí need, ktorý sa nedá obslúžiť.
2. **Nový need debutuje mäkko** — v debutovej vlne má ~`50 %` objemu zavedeného needu (`new_need_debut_ratio 0.50 TUNING`). Unlock je oslava rastu, nie okamžitý trest.
3. **Jedna vlna, jeden nový tlak** — vo vlne, ktorá zavádza nový need, ostatné needy nerastú (držia hodnotu z predošlej vlny). Bez toho sa debut zlúči so všeobecným rastom a hráč nerozozná, čo sa vlastne zmenilo.

Priepustnosť — dve hranice, okolo ktorých sa všetko točí

Budova `Lv1` (`e.g.`): `capacity 3` miesta, `service_time 12 s` (`TUNING`), príchod `90 s`.

Stav budovy	`utilization`	Priepustnosť	Za príchod (90 s)
bez `Worker`-a	`0.33` (`TUNING`)	`3 × 0.33 ÷ 12 ≈ 0.08/s`	**~7**
s `Worker`-om	`1.0`	`3 ÷ 12 = 0.25/s`	**~22**

Dve hranice, nie jedna. Need pod ~7 zvládne aj neobsadená budova. Need medzi ~7 a ~22 zvládne len obsadená. Need nad ~22 nezvládne ani obsadená `Lv1` — treba upgrade alebo Energy. Progresia vĺn nižšie je postavená tak, aby postupne prekračovala obe. (Čísla sú hrubý odhad: pri rovnomernom príchode nie je rad plný od prvej sekundy.)

> **Prečo `12 s`, nie pôvodných `6 s`** (CHANGED 2026-09-29): pri `6 s` zvládla obsadená budova ~45 jednotiek, kým najväčší need prototypu je 35. Obsadená budova teda nikdy nemala problém, problém mala len tá jedna prázdna, a hráč nechal prázdnu tú s najmenším needom — **každá vlna sa dala vyhrať na 5★ jedným rozmiestnením pred štartom**, bez presunov a bez upgradov. Prototyp by netestoval nič. Overené simuláciou (`wave_sim.py`).
>
> **`utilization_empty` (`0.33`) a `service_time` (`12 s`) sú dve najcitlivejšie čísla prototypu — ladiť ich ako prvé a vždy spolu.** Každú zmenu overiť simuláciou proti WR7 a HR2.

Progresia vĺn (`e.g.`, len na rozbehnutie testu)

Rast objemu: vlny 1–3 rastú zámerne strmšie (~+35–100 %), aby sa hráč rýchlo dostal k prvému skutočnému rozhodnutiu; od vlny 4 ≈ `+25 %` na vlnu (`demand_growth_per_wave 0.25 TUNING`), pozastavený v debutových vlnách (W4, W7) podľa pravidla 3.

Stĺpec **„Hostia" je len vizuálny počet postavičiek** — mechanika pracuje s jednotkami needov (hodnotí sa jednotka, nie hosť, §4.4). Jeden hosť nesie v priemere ~2 needy.

Vlna	Hostia	Rooms	Drinks	Food	Relax	Ocean View	Zamýšľaný beat
1	12	8	8	6	—	—	bez bottlenecku; hráč sa učí čítať panel
2	18	12	**16**	10	—	—	prvý bottlenec k: všetko nad ~7, prázdna budova zaostane
3	24	16	18	14	—	—	tlak na dvoch needoch → **unlock `Pool`**
4	26	16	18	14	8	—	debut `Relax` (mäkký), ostatné držia
5	34	20	22	18	16	—	prvý 4-need

Vlna	Hostia	Rooms	Drinks	Food	Relax	Ocean View	Zamýšľaný beat
							bottleneck
6	40	25	28	22	20	—	vrchol pred unlockom → **unlock `Bungalows`**
7	42	25	28	22	20	10	debut `Ocean View` (mäkký), ostatné držia
8	52	31	35	28	25	16	plná zostava, finále prototypu

Mini-event `Sudden rush` je vo **vlne 2** (§4.6) — nikdy nie v debutovej vlne (W4/W7), inak by sa dva nové podnety zliali do jedného.

Čo ktorá vlna testuje — stĺpec „bez zásahu" je výsledok simulácie, keď hráč pred vlnou raz rozmiestni `Worker`-ov a potom nespraví nič (žiadny presun, žiadny upgrade, žiadna Energy):

Vlna	Bez zásahu	Čo sa deje	Čo z hráča dostaneme
1	Perfect	needy nízko; dobrý plán vopred stačí	rozumie vôbec, čo `Worker` robí?
2	`4★`	**mini-event** `Sudden rush` na neobsadenú budovu; presun ju dotiahne	prečíta z panelu, kam ísť? je presun uspokojivý?

Vlna	Bez zásahu	Čo sa deje	Čo z hráča dostaneme
3	4★	dva needy blízko stropu obsadenej budovy	**je alokácia rozhodnutím?** hlavná otázka
4	Perfect	debut `Relax` — zámerne mäkký (pravidlo 2 a 3)	je unlock oslava, nie trest?
5	4★	štyri budovy, traja `Worker`-i, všetko nad ~7	drží napätie, keď je budov viac než pozornosti? **stačí deficit 1?**
6	4★	needy nad stropom `Lv1` — bez upgradov a presunov to nejde na 5★	cíti hráč, že upgrade bol zaslúžený následok?
7	5★	debut `Ocean View` — malý objem, vyššia váha	prečíta, že malé číslo môže bolieť viac? alebo je to mätúce?
8	3★	všetko nad stropom; treba upgrady **aj** rotáciu, prípadne Energy	zvládnuteľné, alebo zahltí? chce pokračovať?

Plne pripravený rezort (všetky budovy `Lv3`) dosiahne v simulácii 5★/Perfect v každej vlne bez Energy — **HR2 drží**.

Vlny 2 a 3 nesú váhu celého testu alokácie — sú to jediné dve pred prvým unlockom, a obe bez zásahu končia na 4★, takže hráč, ktorý reaguje, cíti rozdiel. Vo vlne 2 to zabezpečuje mini-event; bez neho by pasívna hra dala Perfect.

Nástroj pre ekonóma: `wave_sim.py` (companion súbor) — parametre na začiatku, spustiť `python3 wave_sim.py`. Počíta presne model miest a čakania z §4.4, vrátane mini-eventu. Vypíše obe tabuľky vyššie a označí porušenie WR7 aj WR8. Každá zmena čísel wave systému sa má pred buildom overiť ním. Simulácia zámerne nemodeluje presuny počas vlny — počíta najhorší rozumný prípad (pasívneho hráča).

Príjem z vlny (rozhodnuté 2026-09-29)

```
Coins = jednotky vlny × coins_per_unit × coins_rating_mult[rating]
```

`coins_per_unit` = `16` (`e.g.`), násobiče `1★ 0,4 · 2★ 0,6 · 3★ 0,8 · 4★ 1,0 · 5★ 1,2 · Perfect 1,3` (`TUNING`). Odmena **rastie s veľkosťou vlny sama** — netreba tabuľku odmien na každú vlnu, ladí sa jedno číslo.

Bezpečné voči zneužitiu: vlna vždy dobehne a nikto neodíde neobslúžený, takže „jednotky vlny" = vždy celý objem vlny. Hráč nezíska viac tým, že hrá horšie alebo dlhšie — zručnosť do odmeny vstupuje len cez rating.

Prečo 16: spätne odvodené z cien. Overené simuláciou pri pasívnej hre (bez presunov):

Obdobie	Coins (pasívne)	Stačí na
vlny 1–3	~1 900	`Pool` (1 200) + jeden upgrade `Lv2` (500)
vlny 4–6	~3 900	`Bungalows` (2 500) + dva upgrady `Lv2`

Aktívny hráč (vyšší rating) má viac. Slabý hráč si unlock kúpi o vlnu neskôr — to je legitímny stav (sprístupnené, nekúpiteľné) a WR9 zaručí, že vlny fungujú aj bez neho. Ak ekonóm zmení ceny, musí zmeniť aj `coins_per_unit`.

Unlocky (`e.g.` ceny, rozhodnuté gaty)

Oba sú dvojkrokové: **eligibility gate sprístupní, `Coins` kúpia.** V prototype je gate **dokončenie konkrétnej vlny**, nie prah `Resort Value` — skriptované vlny zaručia, že unlock nastane u každého playtestera na tom istom beate, takže sa otázka „vytvorí rozšírenie chuť pokračovať?" testuje bez šumu. V MVP sa oba gaty prepnú na `Reputation` prah, zvyšok flow ostáva rovnaký.

Unlock	Gate	Cena (`e.g.`)	Čo pridá
`Pool`	dokončenie `Wave 3`	`1,200 Coins`	need `Relax` + 1 `Worker`

Unlock	Gate	Cena (e.g.)	Čo pridá
Bungalows	dokončenie Wave 6	2,500 Coins	need Ocean View + 1 Worker

Upgrade v prototype (rozhodnuté 2026-09-29)

3 levely, mení sa len počet miest. worker_slots sú v prototype fixne 1 a base_quality je MVP — upgrade teda nemení nič iné. Rýchlosť obsluhy (service_time) upgrade **nemení**; zrýchlenie patrí Energy.

Level	capacity (miesta)	Cena (e.g.)	Zvládne za príchod s Worker-om	bez Worker-a
1	3	—	~22	**~7**
2	5	500 Coins	~37	**~12**
3	7	1,200 Coins	~52	**~17**

Na čo upgrade v prototype slúži — dve veci naraz: (1) obsadená budova na Lv1 zvládne ~22, ale od vlny 6 sú needy nad tým — upgrade **dvíha strop** obsadenej budovy; (2) budova na Lv3 zvládne ~17 aj bez obsluhy — upgrade rozhoduje, **ktorú budovu si hráč môže dovoliť nechať bežať samu**. Upgrade a alokácia sa tak spájajú do jedného rozhodnutia.

Prečo len miesta: (1) rýchlosť je páka Energy, a keby ju menil aj upgrade, dve páky by robili to isté; (2) počet miest je **viditeľný** (3 → 5 → 7 stoličiek) a pomáha hráčovi rozoznať „primalá" od „neobsadená" — neviditeľná rýchlosť by to rozbila. Prečo len 3 levely: pri 8 vlnách a 5 budovách hráč realisticky stihne najviac dva upgrady na budovu; vyššie levely by väčšina testerov nevidela.

Artist: 3 vizuálne verzie × 5 budov = **15 assetov**; počet miest musí byť z budovy vidieť.

> ⚠ **Chýba:** koľko Coins dáva vlna. Bez toho sa nedá overiť, či sú upgrady na Wave 6 dostupné, a celá progresia stojí na tom, že sú. Tuning pre ekonóma — viď OPEN v Decision Logu.

4.9 Edge cases

Case	Behaviour
Zatvorenie hry počas `WAVE_ACTIVE`	Vlna prejde do `WAVE_PAUSED`: **všetky timery zamrznú, stav sa uloží**. Nič nebeží, nič sa nezhoršuje. Po návrate hráč pokračuje presne tam, kde skončil.
Offline návrat	Nič sa negeneruje; banked Energy nezmenená.
`energy = 0` + Speed Up tapnutý	No-op, ponúkne earn path; služba beží ďalej.
Prázdny need list / vlna bez bottlenecku	Žiadny warning, žiadne nútené CTA; dashboard pokojný.
Vyprší `wave_timer`, v radoch ešte sú jednotky	Vlna **nekončí**: zatvorí sa príchod a začne dobeh. Vlna sa uzavrie, keď sú všetky rady prázdne.
Dobeh trvá dlho (budova bez `Worker`-a, veľký rad)	Beží ďalej, kým rad nie je prázdny — vždy konečne, lebo prázdna budova nikdy nie je na nule. Hráč môže aj v dobehu presúvať a boostovať.
Jednotka čaká veľmi dlho	**Neodíde.** Jej príspevok klesne na `unit_floor` (`e.g.` 25 %) a tam ostane. Odchod nespokojných sa zobrazí až na `Wave Complete`.
Všetko čakalo dlho	`Wave Satisfaction` = `wave_floor` (`e.g.` 40 %) → 1★ a nejaké `Coins`. **Nikdy prázdny výsledok.**
Energy boost beží a appka ide na pozadie	Vlna prejde do `WAVE_PAUSED`, **zamrzne aj zostávajúci čas boostu**. Po návrate pokračuje.
Energy boost druhýkrát na tú istú budovu v tej istej vlne	Nedostupné — 1 boost na budovu za vlnu; `Speed Up` na tej budove sa už neukáže.
Presun `Worker`-a v `WAVE_PAUSED`	Nie je možný — v pauze hráč nie je v hre.

Case	Behaviour
Presun `Worker`-a v `AWAITING_WAVE`	Povolený. Rozmiestnenie platí od `Start Wave`.
Presun `Worker`-a, kým na budovách sedia hostia	Zmena rýchlosti platí okamžite aj pre rozbehnuté obsluhy — cieľová budova zrýchli, zdrojová spomalí, nikto nie je „vyhodený" z miesta.
Energy boost začne alebo skončí, kým hostia sedia	To isté — rýchlosť rozbehnutých obsluh sa zmení okamžite.
Presun na budovu, ktorá už má `Worker`-a	V prototype sa neponúka — každá budova má 1 slot a `Worker`-i sú zameniteľní, takže výmena by nič nezmenila. **Presun v prototype = „vyber, ktorú budovu nechám prázdnu"**: hráč zvolí zdrojovú budovu a `Worker` ide na tú jedinú neobsadenú.
`Sudden rush` nastane počas pauzy alebo tesne pred koncom príchodu	Pauza posunie event spolu s celou vlnou. Event má pevný čas v rámci príchodu (`e.g.` 45 s), takže koniec príchodu ho nezruší.
Hráč tapne `Ignore` pri `Sudden rush`	Výzva zmizne, nával príde normálne a obslúži sa pomaly; nižší rating je jediný dôsledok.
Unlock sprístupnený, ale nekúpený	Stavenisko je `available`; ťuk v `AWAITING_WAVE` otvorí S9. Need sa nespawnuje (WR9).
Hráč tapne upgrade počas bežiacej vlny	Nedostupné — upgrade je len medzi vlnami (§5.3.7). UI to nesmie ponúkať ako zašednuté tlačidlo s odmietnutím; jednoducho tam v tom stave nie je.
Rating < 5★	Mäkký tón, breakdown „needs improvement", žiadny „prehral si".

4.10 Acceptance criteria (Ak / Keď / Potom)

AC1 — bottleneck vzniká z radu čakajúcich Ak sú všetky miesta budovy obsadené a jednotky prichádzajú rýchlejšie, než sa uvoľňujú Keď rad čakajúcich prekročí `bottleneck_threshold` a rastie aspoň `bottleneck_hysteresis_s` Potom need vstúpi

do `BOTTLENECK` a vyvolá warning + guest feedback toast. Jednotky sediace na miestach sa do radu nepočítajú.

AC1b — čakanie končí usadením Ak jednotka príde a miesto je voľné Keď si sadne Potom jej čakanie = 0 s a jej príspevok k satisfaction je 100 %, bez ohľadu na dĺžku obsluhy.

AC2 — presun `Worker`-a vyrieši bottleneck viditeľne Ak je need v `BOTTLENECK` Keď hráč cez S3 presunie `Worker`-a na príslušnú budovu Potom sa obsluha budovy okamžite zrýchli (`utilization 0.33 → 1.0`), need prejde do `RECOVERING` — warning sa **v tej istej interakcii** zmení na pozitívny signál a spustí sa feedback toast. Do `OK` prejde, až keď rad klesne pod prah. Presun je **zadarmo** a nevyžaduje potvrdenie.

AC2b — presun má cenu inde (nulový súčet) Ak hráč presunie `Worker`-a z obsadenej budovy Keď presun prebehne Potom pôvodná budova okamžite klesne na `utilization 0.33` a je **viditeľne bez obsluhy** — hráč musí vidieť, čo obetoval, nie len čo získal.

AC2c — prázdna budova nie je mŕtva Ak je budova bez `Worker`-a Keď jej need dostane demand Potom služba beží ďalej spomalene (`e.g. 0.33`), nikdy nie na nule — need pod `e.g. 15` jednotiek za vlnu sa dá obslúžiť aj bez `Worker`-a.

AC2d — rozlíšenie „malá" vs. „neobsadená" Ak má budova rastúci rad Keď sa hráč pozrie na S2 alebo na budovu Potom vie rozoznať, či je príčina chýbajúci `Worker` (→ presun, zadarmo, hneď) alebo nízka `capacity` (→ upgrade, `Coins`, až po vlne). **Toto je hlavná testovaná otázka §4.1** — ak to build nevie ukázať, prototyp nemá čo merať.

AC3 — vlna sa končí prázdnymi radmi, nie časovačom Ak je príchod uzavretý (uplynul `wave_timer`) Keď posledný need dosiahne `backlog = 0` Potom vlna vstúpi do `WAVE_RESOLVING`. Kým je v niektorom rade aspoň jedna jednotka, vlna beží.

AC4 — `wave_timer` zatvára príchod Ak vlna beží a v radoch sú jednotky Keď `wave_timer` dosiahne 0 Potom **prestane prichádzať nový demand**, vlna pokračuje dobehom a hráč môže ďalej presúvať `Worker`-ov aj boostovať. Žiadna jednotka nie je „stratená" ani neobslúžená.

AC4b — podlahy platia Ak všetky jednotky vlny čakali nad `e.g. 30 s` Keď sa vlna uzavrie Potom `Wave Satisfaction = wave_floor` (`e.g.` 40 %), rating `1`★, a vlna vyplatí nenulové `Coins`. Neplatí to naopak: vlna, v ktorej hráč nespravil nič, **nesmie** dosiahnuť `3`★.

AC4c — Energy boost vo vlne Ak je budova v prevádzke, hráč má `energy ≥ 3` a na tejto budove v tejto vlne ešte neboostoval Keď tapne `Speed Up ⚡3` Potom sa `service_time` budovy na `15 s` zníži na polovicu, `energy` klesne o 3, `Speed Up` na tejto budove zmizne do konca vlny, a po 15 s sa `service_time` vráti na normál.

AC4d — príprava pred vlnou Ak je hra v `AWAITING_WAVE` Keď hráč presunie `Worker`-a a potom tapne `Start Wave` Potom vlna začne s novým rozmiestnením; rozmiestnenie z konca vlny sa zachová aj do ďalšej.

AC5 — pauza zachová stav Ak je vlna v `WAVE_ACTIVE` s čiastočným progresom Keď sa appka na ľubovoľne dlho presunie na pozadie a znovu otvorí Potom sú všetky počítadlá, timery a rozmiestnenie `Worker`-ov presne také, aké boli; nič nepokročilo, nič sa nezhoršilo.

AC6 — rating vysvetľuje sám seba Ak je vlna dokončená Keď sa ukáže výsledková obrazovka Potom obsahuje aj `What went well`, aj `Needs improvement`, a odporúčaný upgrade zodpovedá needu, ktorý bol v `BOTTLENECK` najdlhšie.

AC7a — `Pool` unlock: gate sprístupní, Coins kúpia, pridá sa need aj `Worker` Ak hráč dokončil `Wave 3` a má dosť `Coins` Keď kúpi `Pool` cez S9 Potom sa `Relax` stane aktívnym needom v ďalších vlnách, `Worker` pool sa zväčší z 2 na 3 (prichádza v balíku s budovou, nekupuje sa zvlášť), a forecast ďalšej vlny odráža nový demand. Ak gate nie je splnený, `Pool` je `locked`; ak je splnený ale chýbajú `Coins`, je `available` ale nekúpiteľný.

AC7b — `Bungalows` unlock: to isté, druhý raz Ak hráč dokončil `Wave 6` a má dosť `Coins` Keď kúpi `Bungalows` cez S9 Potom sa `Ocean View` stane aktívnym needom, `Worker` pool sa zväčší z 3 na 4, a S9 sa použije **druhýkrát s tým istým layoutom** — obrazovka nie je jednorazová.

AC7c — nový need debutuje mäkko Ak sa práve odomkla nová budova Keď príde nasledujúca vlna Potom má nový need výrazne nižší objem než zavedené needy a objem zavedených needov oproti predošlej vlne **nestúpne** — hráč vidí jednu novú vec, nie dve naraz.

AC8 — mini-event má vždy primárne riešenie a nepotrebuje Energy Ak vo vlne 2 nastane `Sudden rush` a hráč má `energy = 0` Keď sa event spustí Potom zasiahne práve neobsadenú budovu, ponúkne `Move Worker here`, a po presune sa nával obslúži bez Energy. Vlna sa počas výzvy nepauzuje.

4.11 Čo NIE je v prototype

Reputation · Materials · Gems · VIP hosť · Guest Request Board · Service Quality Choice (Energy Mode B) · klub · eventy (okrem 1 mini-eventu) · IAP · daily loop.

Zo staff modelu nie je v prototype: menovaní `Manager`-i a `manager_multiplier` · zbieranie, rarity a leveling · viac než 1 `worker_slot` na budovu · klesajúci výnos z ďalšieho `Worker`-a · `base_quality` z upgradov (`Lv4+`). Prototyp má **spodnú vrstvu** MVP modelu, nie celý.

4.12 Exit criteria — kedy prejsť na MVP

Pred playtestom (QA): `wave_sim.py` s parametrami buildu neukazuje porušenie WR7 ani WR8.

Prototyp uspel, ak playtest ukáže:

- Hráč **sám** presunie `Worker`-a na bottleneck skôr, než mu to hra povie (rozumie loopu).
- **Alokácia je rozhodnutie, nie lookup** — hráč aspoň raz zaváha, kam `Worker`-a poslať, a vie po session povedať, prečo sa rozhodol tak, ako sa rozhodol. Ak vždy existovala zjavná odpoveď, deficit alebo asymetria medzi budovami je primalá.
- **Hráč rozlíši „budova je malá" od „budova je prázdna"** a podľa toho volí upgrade vs. presun. Ak háda, model je správny, ale nečitateľný — a to treba vyriešiť pred MVP.
- Nikto sa nepokúša ťahať jednotlivých hostí prstom (paradigma „systém" prešla).
- Dokončenie vlny + upgrade pôsobí satisfying, nie ako povinnosť.
- Hráč po session vie povedať, čo zlepší nabudúce (príčina→následok drží).
- Odomknutie `Pool` vyvolá chuť pokračovať, nie „ďalšia vec navyše".
- **Druhý unlock (`Bungalows`) drží ten istý efekt** — chuť pokračovať nevydržala len na prvý raz. Ak druhý unlock pôsobí ako rutina, je to signál pre obsahový rytmus, nie chyba budovy.

Metriky na inštrumentáciu: `wave_complete_rate`, `worker_move_before_prompt_rate`, `worker_moves_pre_wave` a `worker_moves_in_wave` **oddelene** (v prvých vlnách je veľa presunov pred štartom a málo počas vlny znakom, že hráč pochopil; nula v oboch znamená, že nepochopil), `drain_duration_s` (dĺžka dobehu — dlhé dobehy = hráč nereaguje), `rating_distribution` (ak `3★+` dostávajú aj hráči bez presunov, podlahy sú privysoko), `upgrade_take_rate`, `session_length`, subjektívny „chcem ešte jednu vlnu". **Nemerať:** Energy usage (nie je predmet testu).

> **Session length: 8 vĺn je realisticky 12–20 minút**, nie cieľových 3–7. Buď sa playtest rozdelí na viac sedení, alebo sa vedome akceptuje, že **session shape sa týmto prototypom netestuje čisto** a overí sa až neskôr na kratšom výseku. Nie je to chyba buildu — je to dôsledok toho, že dva unlocky potrebujú miesto.

---

5. MVP

5.1 Čo MVP pridáva na overený prototyp

MVP = prototyp (validovaný) **plus**:

1. **VIP vrstva** — Preview → Request Board → VIP Complete (S6–S8).
2. **Reputation** ako threshold unlock mena.
3. **Obsah** — VIP typy hostí, `Specialist`-i (`Butler`, `Captain` — rosterový predpoklad pre časť VIP requestov; chýbajúci znižuje strop kvality, neblokuje — nepriraďuje sa na request), a `Spa` ako **VIP budova** (bez wave needu, §5.4).
4. **Materials + Gems** do ekonomiky — `Materials` pre VIP a `Lv4–5`, `Gems` len na trvalé prémiové nákupy (§5.6).
5. **Monetizačné háčiky** (bez detailného ladenia) — rewarded video, prvé IAP miesta.
6. **Energy Mode B** (Service Quality → Signature).
7. **Dvojvrstvový staff model** — `Manager` (najíma sa za `Coins`, násobič budovy) nad `Worker`-mi z prototypu, plus najímanie ďalších `Worker`-ov so stropom. Viď §5.3.
8. **Upgrade rady budov do Lv5** — `Lv1–3` sú presne tie z prototypu, MVP pridá `Lv4` (slot + kvalita) a `Lv5` (miesta + slot + kvalita). Viď §5.3.6.

5.2 Scope v kusoch (producent)

Delta oproti prototypu, nie celkové počty od nuly:

Prvok	MVP počet	Delta vs. prototyp	Zdroj
Typy hostí	5	+4 VIP (`Travel Blogger`, `Wedding Couple`, `Investor`, `Influencer`)	Verzia II/IV
`Manager`	5	**nové** — `Receptionist`, `Chef`, `Bartender`, `Lifeguard`, `Bellhop`; jeden na svoju wave budovu, **najíma sa za `Coins`** (jednorazovo), násobí výkon budovy a je	§5.3.2

Prvok	MVP počet	Delta vs. prototyp	Zdroj
		rosterovým predpokladom pre VIP requesty tej budovy. Žiadne zbieranie ani rarity (PARKED na po-MVP).	
`Specialist`	2	**nové** — `Butler`, `Captain`; bez budovy, **najíma sa za `Coins`**, vo vlnách nerobí nič, len rosterový predpoklad pre VIP requesty.	§5.3.2
`Worker`	pool (strop `TUNING`)	prototyp: +1 s každou wave budovou; **MVP pridáva najímanie za `Coins`** so stropom `80 %` slotov (`TUNING`).	§5.3.5
Level budov	Lv1–Lv5	`Lv1–3` bez zmeny z prototypu; **+`Lv4`, `Lv5`** (sloty, kvalita)	§5.3.6
Budovy	6	+1 (`Spa`) — **VIP budova**: bez wave needu a bez slotu pre `Worker`-a	§5.4
Need typy	5	bez zmeny oproti koncu prototypu (VIP ide cez requesty, nie needy)	Verzia IV
Requesty na VIP	3 + 1 bonus	nové	Verzia II/III
Rating	1–5★ + Perfect Stay	rozšírené o VIP variant	Verzia III

Prvok	MVP počet	Delta vs. prototyp	Zdroj
Meny	`Coins`, `Energy`, `Reputation`, `Materials`, `Gems`	+3	Verzia II
Screeny	9 (S1–S9)	+3 (S6, S7, S8)	UI handoff

5.3 Model bottlenecku: budova × staff × Energy

Toto je nosná sekcia MVP. Určuje, ako vzniká problém vo vlne a akými pákami sa rieši. Rozhodnuté 2026-09-29.

5.3.1 Jedna rovnica

```
None
fill_rate(služba) = capacity(level budovy)        ← STROP
                  × utilization(pridelení Workers) ← KOĽKO ZO STROPU BEŽÍ
                  × manager_multiplier             ← KVALITA VEDENIA
                  × energy_boost                   ← DOČASNE, VOLITEĽNE
```

V modeli miest (§4.4) to znamená: čas obsluhy jedného hosťa = `service_time ÷ (utilization × manager_multiplier × energy_boost)`. `Manager` (`e.g. ×1.2`) teda obsluhu **skracuje** na každom mieste budovy — s `Worker`-om aj bez neho.

Budova určuje strop. Staff určuje, koľko z toho stropu je reálne zapnuté. Z toho plynie celý zvyšok: `Restaurant Lv5` bez `Worker`-ov je poloprázdna hala; `Restaurant Lv1` s `Worker`-om je stále malá budova, do ktorej sa viac ľudí nezmestí. Hráč musí držať obe veci v rovnováhe — a otázka „je moje slabé miesto budova, alebo ľudia?" je práve to rozhodnutie, ktoré má MVP testovať.

5.3.2 Dve vrstvy staffu

Vrstva	Čo to je	Ako sa získava (MVP)	Kde vstupuje
`Manager`	menovaný špecialista, jeden na wave budovu	**najíma sa za `Coins`** (`e.g.` 800), jednorazovo	`manager_multiplier` (`e.g. ×1.2`) + VIP roster tej budovy
`Specialist`	menovaný VIP pracovník **bez**	**najíma sa za `Coins`** (`e.g.` 1 000), jednorazovo	len VIP roster

Vrstva	Čo to je	Ako sa získava (MVP)	Kde vstupuje
	budovy (`Butler`, `Captain`)		
`Worker`	anonymný pracovník, zameniteľný	+1 s každou wave budovou; ďalší za `Coins` so stropom (§5.3.5)	`utilization`

Prečo najímanie za `Coins` (rozhodnuté 2026-09-29): je to skutočné rozhodnutie z jednej peňaženky — *upgrade budovy, `Manager`, alebo ďalší `Worker`?* Každá z troch vecí pomôže inak: upgrade pridá miesta, `Manager` zrýchli všetko, čo budova robí (s `Worker`-om aj bez neho), a `Worker` pomôže tam, kam ho hráč pošle. `Manager` navyše odomkne vyšší strop kvality pri VIP. Keby `Manager` prišiel zadarmo s budovou, bola by to len skrytá zmena `service_time` bez rozhodnutia — a neskoršie zbieranie (rarity, PARKED) by nemalo na čo nadviazať.

Prečo samostatný pojem `Specialist`: pre programátora je `Manager` dátová štruktúra *patrí k budove X a násobí jej výkon o Y*. `Butler` k žiadnej budove nepatrí a nič nenásobí — pod jedným názvom by vznikla výnimka „`Manager`, ktorý nie je `Manager`" (rules §6).

Kde sa najíma: v `AWAITING_WAVE` — na S5 (ponuka vedľa upgradu) alebo tapnutím na budovu. Počas vlny sa nenajíma (rovnaký dôvod ako pri upgrade, §5.3.7).

`Manager` je násobič, nie spínač. Služba bez manažéra beží pomalšie, nikdy nie vôbec — inak by sa z rostera stal hard gate (rozpor s pilierom 2 a s prístupom „chýbajúci = nižší strop kvality, nie blok", ktorý už platí pre VIP rosterové predpoklady).

Rozptyl manager_multiplier držať úzko (`e.g. 1.0`–`1.4`, `TUNING`). Ak je široký, silný manažér prebije alokáciu a spodná vrstva prestane rozhodovať.

> ⚠ **TERMINOLOGICKÁ ZMENA.** V starších záznamoch Decision Logu znamená slovo `Staff` päticu menovaných rolí (`Receptionist`, `Chef`, `Bartender`, `Lifeguard`, `Bellhop`). Od 2026-09-29 sú to `Manager`-i, `Butler` a `Captain` sú `Specialist`-i, a `Staff` je **zastrešujúci pojem** pre všetky tri vrstvy (rules §6).

5.3.3 Tri páky, tri časové škály

Toto je to, čo ich robí neredundantnými. Bez rozlíšenia riešia všetky to isté („služba nestíha") a hráč nevie, ktorú siahnuť.

Páka	Kedy	Trvanie	Rieši	Čo NEVIE
Upgrade budovy	medzi vlnami	trvalé	*„táto služba je chronicky slabá"*	nepomôže s tým, čo horí teraz
Alokácia Workers	počas vlny	do konca vlny	*„tento problém je akútny"*	nulový súčet — odobratím inde vytvoríš druhý problém
Energy	počas vlny, na sekundy	okamih	*„nestíham o kúsok"*	nedvíha strop

5.3.4 Presun Workers je ZADARMO (ochrana HR1/HR2)

Akcia	Cena	Kedy
Presunúť Workers medzi budovami	**zadarmo, vždy**	kedykoľvek počas vlny
Energy boost na konkrétnu budovu	Energy	keď presun nestačí

Energy nesmie byť cenou za presun Workera. Keby bola, hráč bez Energy by nemohol riešiť bottleneck — Energy by prestala byť boost a stala by sa cenou za základnú akciu. Tým padá HR1 (nič nesmie vyžadovať Energy na spustenie) aj HR2 (Perfect Stay dosiahnuteľný bez Energy) a vzniká death spiral: slabá vlna → málo Energy → nemôžem riešiť ďalšiu vlnu → ešte slabšia.

Hráč bez Energy vlnu **zvládne, len horšie**. S Energy ju zvládne pekne. To je definícia optional boostu — a je to zároveň lepší monetizačný model: predáva sa *lepší výsledok*, nie *možnosť hrať*.

Energy nikdy nedvíha `capacity`, len dočasne zrýchľuje priebeh (Mode A). Preto sa každý bottleneck dá vyriešiť upgradmi a alokáciou.

5.3.5 Čo robí alokáciu skutočným rozhodnutím

Dve obmedzenia, bez ktorých sa model zosype na „nalej všetkých tam, kde horí":

1. `worker_slots` **na budove** — limit, koľko ľudí budova uvezie. Rastie s levelom (§5.3.6).

2. **Klesajúci výnos** — druhý Worker pridá menej než prvý, tretí menej než druhý (TUNING). Rozloženie je takmer vždy lepšie než koncentrácia; to „takmer" je priestor na rozhodnutie.

Ako pribúdajú Worker-i v MVP (rozhodnuté 2026-09-29)

- **S každou novou wave budovou príde 1 Worker** — rovnako ako v prototype.
- **Ďalších si hráč najíma za Coins** (e.g. 400, každý ďalší o e.g. 200 drahší, TUNING), len v AWAITING_WAVE. **Pri najatí hráč vyberie budovu s voľným slotom**, kam Worker nastúpi.
- **Presun pri viacerých slotoch:** od Lv4 sa dá Worker poslať aj na budovu, ktorá už niekoho má, pokiaľ má voľný slot. Prototypové pravidlo „presun = ktorú budovu nechám prázdnu" platí len pri jednom slote na budovu. Worker ani v MVP nikdy nestojí mimo budovy.
- **Strop:** najať sa dá najviac worker_cap_ratio × súčet slotov, zaokrúhlené nadol (worker_cap_ratio 0.8, TUNING). Keď je hráč na strope, ponuka ukáže *„Upgrade buildings to open more shifts"* namiesto tlačidla.

Hard pravidlo (spec): Worker-ov je vždy menej než súčet slotov. Deficit je jadro alokácie — keby sa dali obsadiť všetky sloty, presúvanie by stratilo zmysel.

Prečo 80 % a nie pôvodných ~60 % (CHANGED 2026-09-29): prototyp končí s 5 slotmi a 4 Worker-mi, teda na 80 %. Pri 60 % by hráč na začiatku MVP mal nad stropom a model by sa zlomil hneď na štarte. 80 % sedí na koniec prototypu (5 → 4) a pri plne vylepšených budovách (15 slotov → 12) nechá každú budovu obsadenú, ale nie naplno.

Prečo najímanie a nie automatický prírastok: automaticky (60 % slotov) by Worker-i „vyskakovali" pri zaokrúhľovaní v nečakaných chvíľach a upgrady by dostali skrytú hodnotu — oplatilo by sa upgradovať nie preto, že budova nestíha, ale aby pribudol Worker (proti pilieru 4). Pri najímaní hráč presne vie, čo kúpil a prečo.

5.3.6 Upgrade rada Lv1 → Lv5 (e.g., rozhodnuté 2026-09-29)

Lv1–3 sú presne tie z prototypu. MVP na ne len nadviaže. Slot level sa preto posúva z pôvodne navrhovaného Lv3 na Lv4 (CHANGED 2026-09-29). Všetko v absolútnych miestach, nie v percentách — jednotne s prototypom.

Level	capacity (miesta)	worker_slots	base_quality	Cena (e.g.)	Čo hráč cíti
1	3	1	—	—	východisko
2	5	1	—	500 Coins	„zvláda viac ľudí"

Level	capacity (miesta)	worker_slots	base_quality	Cena (e.g.)	Čo hráč cíti
3	7	1	—	1,200 Coins	„poriadna budova"
4	7	**2**	**+tier**	2,500 Coins + Materials	„môžem sem poslať posilu — a hostia sú spokojnejší"
5	**10**	**3**	+tier	5,000 Coins + Materials	„vlajková loď"

- **Lv4 je zámerne bez nových miest** — je to *slot level*. Striedanie „viac miest" a „nová možnosť" z pôvodného zámeru ostáva, len o level vyššie. Hráč z prototypu sloty nepozná, takže v MVP prídu ako nová vec, nie ako zmena známej.
- **base_quality od Lv4** je poistka proti pasci modelu: budova bez Worker-ov by inak bola mŕtva a upgrade by pôsobil ako vyhodené peniaze. Od Lv4 prispieva k satisfaction aj pri slabom obsadení. Je to zároveň prirodzené miesto pre Mode B (Energy spec §1, OPEN).
- **Lv4 a Lv5 stoja aj Materials** (§5.6) — najvyššie levely sú tak viazané na VIP hru.
- **Lv5 musí byť viditeľný z odstupu** — vizuálny milestone. Je to koniec rady v MVP.

Artist: Lv1–3 z prototypu platia ďalej; MVP pridá Lv4 a Lv5 pre 5 wave budov (**10 assetov**) + Spa (vlastná rada, počet levelov OPEN — VIP budova nepotrebuje sloty).

5.3.7 Upgrade prebieha VÝHRADNE medzi vlnami (rozhodnuté 2026-09-29)

Počas bežiacej vlny sa nedá upgradovať — ani okamžite za Coins, ani so stavebným timerom.

Prečo nie okamžitý upgrade	Prečo nie timer
Zabije dve z troch pák. Hráč s Coins už nemusí presúvať Worker-ov ani míňať Energy — kúpi capacity a problém zmizne natrvalo. Upgrade by bol lepší než ostatné páky vo všetkom okrem ceny a alokačná hra by sa scvrkla na „mám dosť peňazí?".	**Návrat pre dobehnutý timer, nie pre výsledok.** Stavba na hodiny je presne ten mechanizmus, ktorému §7 pravidiel predchádza.

Prečo nie okamžitý upgrade	Prečo nie timer
Rozbije ekonomický rytmus. `Coins` pochádzajú z dokončených vĺn; upgrade uprostred vlny míňa odmenu za vlnu, ktorá ešte neskončila — a tá odmena bude vyššia práve preto, že si ju upgradom vylepšil. Ťažko laditeľná slučka.	
Mení, čo vlna je. Vlna má byť test *pripravenosti rezortu*. Ak sa rezort dá prestavať uprostred testu, netestuje pripravenosť, ale hotovosť v peňaženke — rytmus „pripravím → otestujem → zlepším" sa mení na „improvizujem za pochodu".	

Priznaný náklad: hráč, ktorý uprostred vlny pochopí „bar je primalý", musí počkať do konca, aby to opravil. Nepovažujeme to za stratu — je to napätie, ktoré ho ťahá do ďalšej vlny — ale je to reálny náklad tohto rozhodnutia.

Miesto pre „konám aj stavebne počas vlny" v modeli už zaplnené je: Energy Mode B (quality boost) dvíha kvalitu služby počas behu. Je dočasný a nemíňa `Coins`, takže upgradu nekonkuruje, ale dopĺňa ho.

> **Status:** rozhodnuté ako „zatiaľ nepovolíme". Ak sa to bude neskôr otvárať, podmienka znie: musí existovať mechanizmus, ktorý zabráni tomu, aby upgrade počas vlny prebil presun `Worker`-a a Energy. Bez neho ostávajú v hre dve mŕtve páky.

5.3.8 Worked example — ako to beží v číslach

Čísla `e.g.`, na ilustráciu mechaniky pre programátora a ekonóma. Nie balancing. „Hosť" v príkladoch = jedna jednotka needu (`Drinks`, `Relax`…), lebo hodnotí sa jednotka, nie hosť (§4.4). Energy boost = `service_time × 0.5` na `15 s` (§4.6).

Scenár A — 10 hostí príde na `Beach Bar Lv1`

`capacity 3` · `service_time 12 s` · `worker_slots 1` · priradený `Worker`: **0**

t	Čo sa deje	Čísla
0 s	10 jednotiek `Drinks` dorazí; 3 k pultu, 7 do radu	`backlog 7`

t	Čo sa deje	Čísla
0–15 s	obsluha beží bez `Worker`-a, `utilization ≈ 0.33`	priepustnosť `3 × 0.33 ÷ 12 ≈ 0.08/s`
hneď	`backlog 7` > `bottleneck_threshold 4` → `BOTTLENECK`	warning + guest toast
15 s	hráč presunie `Worker`-a z `Restaurant` (zadarmo, okamžite)	`utilization 1.0` → `3 ÷ 12 = 0.25/s`, **3×**
15 s	rad sa začne zmenšovať	`BOTTLENECK → RECOVERING`, pozitívny signál
15–51 s	bar dobieha ~9 jednotiek	rad pod prahom → `OK`
paralelne	`Restaurant` ostal bez obsluhy, `Food` začína zaostávať	**nulový súčet — to je zámer**

Vetva 1 (vlna má čas): bar dobehne, hráč vráti `Worker`-a do `Restaurant`. Väčšina čakala v rámci `patience_grace` alebo tesne nad → vysoký rating. „Zvládol som to manažmentom."

Vetva 2 (je neskoro): príde ďalších 10 a ani `0.25/s` nestačí — presun už nepomôže, lebo `capacity 3` je strop. **Tu prichádza Energy**: Mode A skráti `service_time` na `15 s` (`12 s → 6 s`), priepustnosť `0.5/s`. Hráč bez Energy vlnu zvládne, len s horším ratingom.

Po vlne: rating `e.g. 4★`, časť hostí odchádza nespokojná. Na S5 je vidno, že `Beach Bar` bol slabé miesto → upgrade `Lv2`, `capacity 3 → 5`. Nabudúce tých istých 10 s jedným `Worker`-om: `5 ÷ 12 ≈ 0.42/s` — zvládnuté bez Energy, bez presunu, bez obetovania `Restaurant`. **Loop uzavretý.**

Scenár B — dve budovy horia, voľný je jeden `Worker`

Stav po `Pool` unlocku: 4 budovy, 3 `Worker`-i, `t = 45 s` z `90 s`. Voľný `Worker` je v `Reception`, kde je pokoj. Pre ilustráciu asymetrie má `Pool` pomalšiu obsluhu — **v MVP môžu mať budovy rôzny `service_time`; prototyp má jeden spoločný.**

Budova	Lv	capacity	service_time	backlog	bez Worker-a	s Worker-om
Beach Bar	2	5	12 s	**9**	≈ 0.14/s	≈ 0.42/s
Pool	1	3	20 s	**5**	≈ 0.05/s	0.15/s

Možnosť A — Worker na bar: rad 9 zmizne za ~21 s. Pool medzitým obslúži ~1 (backlog 5 → 4). Potom Worker na Pool: zvyšné 4 za ~27 s. **Hotové ~t 93 s.**

Možnosť B — Worker na Pool: rad 5 zmizne za ~33 s. Bar medzitým obslúži ~5 (backlog 9 → 4). Potom Worker na bar: zvyšné 4 za ~10 s. **Hotové ~t 88 s.**

Časy sú takmer rovnaké — rozdiel je v tom, KTO čakal a ako dlho. Z toho plynú tri veci:

1. **Rozhodnutie nie je automaticky ťažké — asymetriu treba zabudovať vedome.** Keby obe budovy mali rovnakú capacity, rovnaký service_time a rovnakú váhu needu, je jedno, čo hráč urobí. Zdroje asymetrie, ktoré model má: rôzne levely, rôzny service_time, **rôzne váhy needov** (Ocean View váži viac), rôzny zostávajúci čas. Ekonóm ich musí ladiť zámerne.
2. **Vzniká heuristika a to je dobre:** *„pošli pomoc tam, kde budova nezvláda sama"* (Pool má takú nízku priepustnosť, že sa sám nedobehne nikdy). Casual hráč sa ju naučí a bude sa cítiť šikovný — ale pri vysokej váhe needu alebo krátkom zvyšnom čase platí iné. Naučiteľné, nie vyriešené.
3. **Hráč to nepočíta, odhaduje** — na mobile, za tri sekundy, palcom. Na paneli preto nemajú byť čísla, ale **naliehavosť**: ktorý rad rastie rýchlejšie a ktorá budova stojí prázdna.

5.3.9 Riziká tohto modelu

Riziko	Prečo	Návrh na zmiernenie
Tri páky sú na mobil veľa	počas vlny sa hráč rozhoduje medzi alokáciou a Energy, medzi vlnami medzi piatimi budovami	vo vlne nech je primárna len alokácia; Energy sa **ponúkne** až keď alokácia nestačí (e.g. backlog nad prahom) — ako ponuka, nikdy ako gate
utilization je neviditeľná veličina	hráč vidí Food 4/10, nie „strop × využitie" — ale musí	UI musí rozlíšiť „budova je malá" od „nie je tu nikto"; je to

Riziko	Prečo	Návrh na zmiernenie
	z toho prečítať, či investovať do budovy alebo do ľudí	najťažšia úloha zadania pre dizajnérku
Rarity manažéri môžu obísť core loop	slot navyše mení kapacitu systému, nie výkon jednotky	sloty ako vzácny a stropovaný bonus (`e.g.` max `+1`, len najvyššia rarita); násobiče môžu byť bežné — viď PARKED v Decision Logu

5.4 VIP vrstva

- VIP je **nadstavba**, nie prerušenie: bežná vlna beží ďalej, VIP je priorita navyše (US16).

- Nikdy povinný: `Accept / Maybe later / Decline` vždy dostupné (US17). Nedostupnosť ≠ trest.

- Detailný management len tu: VIP sa rieši cez Request Board (discrete requesty), bežní hostia nikdy.

- **VIP sa rieši PRIPRAVENOSŤOU, nie priraďovaním staffu** (rozhodnuté 2026-09-17): request sa splní tým, že hráč má vopred správne budovy / dekor / `Materials` / **najatých `Manager`-ov a `Specialist`-ov**. Board **nepoužíva S3 (Worker Allocation)** — akcie sú `Fix` (dodaj chýbajúce) a `Speed Up` (Energy). Iná resolution logika než bežná vlna: VIP testuje „bol som pripravený?", nie „hasím bottleneck teraz".

- **Pripravenosť má dve vrstvy s odlišným správaním** (rozhodnuté 2026-09-28):

Vrstva	Príklad	Správanie
Spotrebná — tvrdá, opraviteľná hneď	`Materials`, dekor	request je `MISSING` → `Fix` ho prepne na `READY`
Rosterová — mäkká, degraduje	najatý `Manager` / `Specialist`, postavená budova (napr. `Spa`)	request **neblokuje**, znižuje jeho `quality_cap`

  Dôvod deliacej čiary: po suroviny hráč „dobehne do skladu" počas pobytu hosťa, Butlera uprostred pobytu nenajme. Tvrdá brána na staff by z VIP spravila checklist („choď si kúpiť Butlera"), mäkká degradácia z toho robí investičné rozhodnutie.

- **Staff pri VIP = rosterový predpoklad, nikdy priradenie.** Request môže vyžadovať rolu — `Manager`-a svojej budovy (napr. `Chef` pre „Chef's special dinner") alebo `Specialist`-u (napr. `Butler`); bez nej sa dá splniť, ale s nižším stropom kvality. Požiadavka musí byť viditeľná na S6 **pred prijatím** a role musia byť znovupoužiteľné naprieč VIP typmi — inak vznikne nákupný príkaz namiesto plánovania.

- **`Spa` je VIP budova** (rozhodnuté 2026-09-29). Vo vlnách nemá need ani slot pre `Worker`-a; pri VIP je **rosterový predpoklad** — requesty ako `Couples Spa` bez nej majú strop kvality (`e.g. max 3★ – no Spa`). Hráč ju kúpi preto, že mu VIP ukázal konkrétny strop (pilier 4). Odomyká sa cez S9, v MVP s `Reputation` gate-om. **Závislosť pre obsah VIP:** aspoň **dva** zo štyroch VIP typov musia mať request, ktorý `Spa` vyžaduje — inak je `Spa` dekorácia. Návrh (`e.g.`): `Wedding Couple` (`Couples Spa`), `Influencer` (`Wellness Content Shoot`). Prečo nie nový wave need: panel by narástol na 6 meračov, `Wellness` by sa bil s `Relax` a unlock by v MVP pridal len ďalšiu prácu, nie nový zážitok. Need sa dá pridať neskôr; odobrať už zavedený need je ťažké.

- Odmena = spike; **`Reputation` je hlavná** (VIP je jej primárny zdroj, US19), + Coins, Energy, **`Materials`**, Resort Value, rare decor. Nesmie znehodnotiť bežné vlny.

- State machine požiadavky: `MISSING` (chýba **spotrebný** zdroj → `Fix`) `→ READY → IN_PROGRESS` (dá sa `Speed Up`) `→ COMPLETED`. „Chýbajúci zdroj" = `Materials`, nie nová surovina (pravidlo §10). `quality_cap` z rosterových medzier je **atribút requestu, nie stav** — request môže byť `READY` a zároveň stropovaný.

- **Energy Mode B pri VIP** sa spúšťa v `READY` (pred štartom) a slúži ako **kompenzácia za nedokonalú prípravu** — vie odkúpiť časť `quality_cap`. Plne pripravený rezort dosiahne Perfect Stay bez Energy (HR2); Mode B nesmie byť jedinou cestou k nemu.

5.5 Resort Value vs. Reputation — zavreté

Rozdelenie (rozhodnuté D1+D2):

	Resort Value	**Reputation**
Rola	status / prestíž, hlavný ukazovateľ	unlock mena, threshold model
Míňa sa?	**nie** — len rastie, nikdy sa neutráca	**nie** — kumulatívne skóre, nikdy neklesá; unlock pri míľnikoch

	Resort Value	**Reputation**
Zdroj	budovy, upgrady, staff, hodnotenia (odvodené)	odmena z vĺn a VIP
Kde na UI	top bar + result screeny (hero číslo)	sekundárne, pri unlock gate-och

Tým prestanú byť „jedna mena s dvoma názvami". Verzia II toto už implikuje (Reputation odomyká, Resort Value je status) — návrh len robí hranicu explicitnou.

Unlock je dvojkrokový: `Reputation` prah **sprístupní** upgrade/oblasť, `Coins` ho **kúpia**. Reputation sa teda nemíňa (drží threshold), Coins áno. V prototype (bez Reputation) gate nahrádza progresný míľnik; transakcia za `Coins` ostáva.

5.6 Meny & materiály

Pravidlo §10: každá mena musí vedieť povedať, čo robí a čo existujúce meny nezvládnu (rozhodnuté 2026-09-29 pre `Materials` a `Gems`):

Mena	**Na čo**	**Odkiaľ**	**Čo nezvládnu ostatné**
`Coins`	upgrady `Lv2–5`, unlocky, najímanie `Manager`-ov, `Specialist`-ov a `Worker`-ov	vlny (vzorec §4.8), VIP	— základná mena
`Energy`	optional boost; MVP pridá Mode A pri VIP a Mode B	drop z vĺn a VIP podľa ratingu, rewarded video	— viď Energy spec
`Reputation`	eligibility gate na unlocky (sprístupní, nemíňa sa; §5.5)	hlavne VIP	— status + gate
`Materials`	**VIP spotrebné** (`Fix`, napr. `2x Mangoes`) **+ upgrady `Lv4` a `Lv5`** (spolu s `Coins`)	dokončené VIP + malý drop z vĺn na `4★` a vyššie (`e.g.` 1–3 kusy, `TUNING`)	`Coins` rastú automaticky s vlnami; `Materials` **viažu najvyššie levely na VIP hru** — hráč, ktorý VIP ignoruje, sa

Mena	Na čo	Odkiaľ	Čo nezvládnu ostatné
			nedostane na vlajkové lode
`Gems`	**len trvalé prémiové nákupy** — dekor, balíčky (§5.7)	IAP (+ občas odmena `e.g.` za prvý Perfect Stay)	háčik pre monetizáciu, ktorý nesiaha na core loop

`Gems` nič nezrýchľujú a nedajú sa meniť na Energy. V hre nie je čo zrýchľovať (upgrady sú okamžité, vlna sa zrýchliť nedá) a zrýchlenie služieb už robí Energy — `Gems` na zrýchlenie by boli jej kópia. A rules §11 chcú IAP ako trvalú investíciu, nie dočasný boost.

5.7 Monetizačné háčiky (miesta, nie ladenie)

Kým nie je overený core, monetizáciu **neladíme** — len necháme miesta, kam sa zavesí:

- Rewarded video: 2× tips, Energy top-up, extra daily chest, guest refresh (dobrovoľné, výhodné — §11).
- IAP ako trvalá investícia (za `Gems`): `Premium Staff Pack` (najatie `Manager`-a / `Specialist`-u bez `Coins`), `Villa Upgrade Pack`, `Decoration Set` — nie dočasný boost. *Pozor:* `Premium Staff Pack` nesmie obísť strop `Worker`-ov (§5.3.5) — predáva sa pohodlie, nie viac, než smie mať nemonetizujúci hráč.
- Cieľové metriky na neskôr: conversion, ARPPU, D7 retention. Žiadny nátlak, žiadny trest za neplatenie.

5.8 Rozhodnutia pred MVP buildom — zavreté

Zavreté (viď Decision Log): Resort Value vs. Reputation (§5.5), Reputation = threshold model, Request Board = VIP only, Energy cap / refund / max boosts, Wave/VIP Complete = jeden template, Home VIP panel = podmienený prvok, a 2026-09-29: `Spa` = VIP budova, `Manager` / `Specialist` najímanie za `Coins`, rast `Worker`-ov so stropom `80 %`, upgrade rada `Lv1–5`, úloha `Materials` a `Gems`.

Otvorené dizajnové body MVP (neblokujú prototyp, musia padnúť pred návrhom MVP):

- **Energy Mode B vo vlne** — pôvodne sa viazal na priradenie menovaného staffu, ktoré zaniklo.
- **VIP rating thresholdy** — ostali na starej škále (`3★ 50 · 4★ 70 · 5★ 90`), wave škála sa zmenila; zjednotiť alebo vedome ponechať odlišné.
- **Upgrade rada `Spa`** — koľko levelov má VIP budova bez slotov a čo mení.

Tuning pre ekonóma (na dátach z MVP): kedy zapnúť Energy cap, míľniky Reputation unlockov, ceny najímania, drop `Materials`, satisfaction krivka.

5.9 Exit criteria — kedy je MVP hotový na ďalší krok

- Core wave loop drží aj s obsahom (nie je nudný po 5 vlnách).
- VIP pôsobí ako príležitosť a spike, nie ako prerušenie či trest.
- Monetizačné háčiky sedia bez toho, aby čokoľvek trestali.
- Návrat po neaktivite je pozitívny (comeback reward, žiadna degradácia).
- Hráč po prvej session vie povedať, prečo sa chce vrátiť.

---

6. Ďalší krok (po MVP)

Zámerne neurčené — rozhodneme po dátach z MVP. Typicky sem patrí: obsahové škálovanie (viac hostí/budov/eventov), skutočné ekonomické balancing (až teraz sa ladia čísla), live-ops vrstva (eventy, season), a soft-launch príprava. Nič z toho neriešime, kým prototyp a MVP nedokázali, že core loop stojí.

---

7. Companion dokumenty

- **First Session User Story Walkthrough** — prechod prvou session cez user stories (US1–US21).
- **Energy System Spec** — hard rules, params, state machine, edge cases, ekonóm 2×/0.5×.
- **UI Screens Handoff** — S1–S9, formát pre designerku, čo nenavrhovať.
- **Decision Log** — chronologický záznam rozhodnutí a zmien (s dátumami).
- **Project rules** — piliere, non-goals, terminológia, decision log formát.

Tento dokument ich needuplikuje — odkazuje na ne. Pri handoffe ísť vždy s celou trojicou.